



PROJECT: DESIGNING A SERVERLESS MICROSERVICE APPLICATION BACKEND SOLUTION FOR A JUMIA-LIKE APP ON AWS.

PROJECT AGENDA: In this project, we will design the **architecture** for a client moving from an application **hosted on their own-premises infrastructure** to an application hosted on **AWS**. This client has already **migrated its downstream services (Fulfilment Service, Accounting Service, Inventory Service, Payment Service e.t.c)** to AWS, but is looking for advice on how to move its **order service application code**, which currently includes **fulfillment, accounting, and inventory services code** and is acting as one **monolithic codebase**.

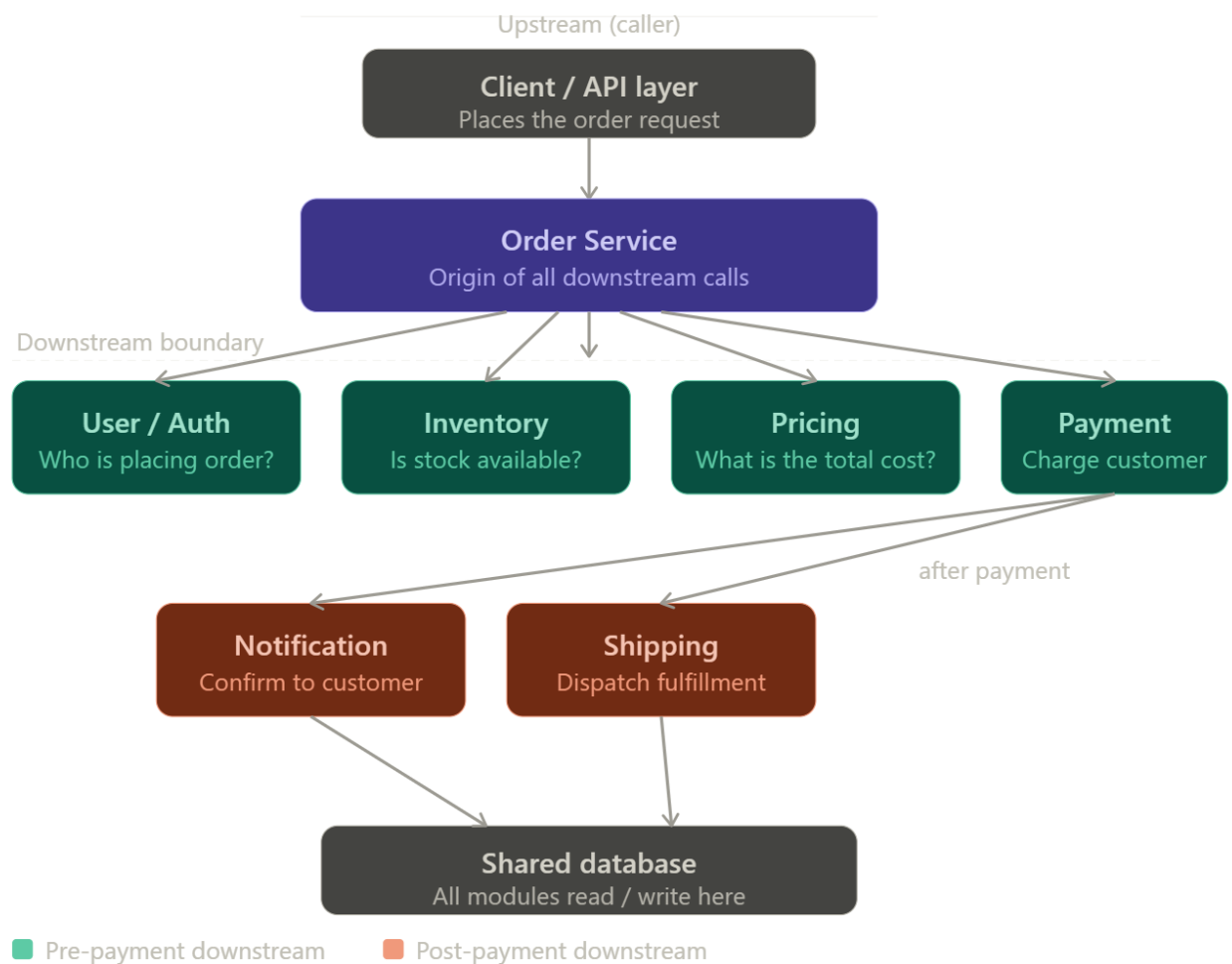
“A **monolithic application** is a single program where **all the parts of your application code are bundled together and run as one unit**”. For the business **order service application code**, the **fulfilment, payments, notifications, inventory, accounting, and so on code** are all in one **codebase** and **deployed together on one server**. If one part of the **code breaks or needs an update**, we will have to **redeploy** the whole **codebase**.

This client wants this **order service application** to be more **reliable** and **resilient**, so that if **one part of the application fails**, it does not impact the **other processes running**. The current

problem the business is having is that if they have a **single point of failure**, it breaks the **entire order service process**.

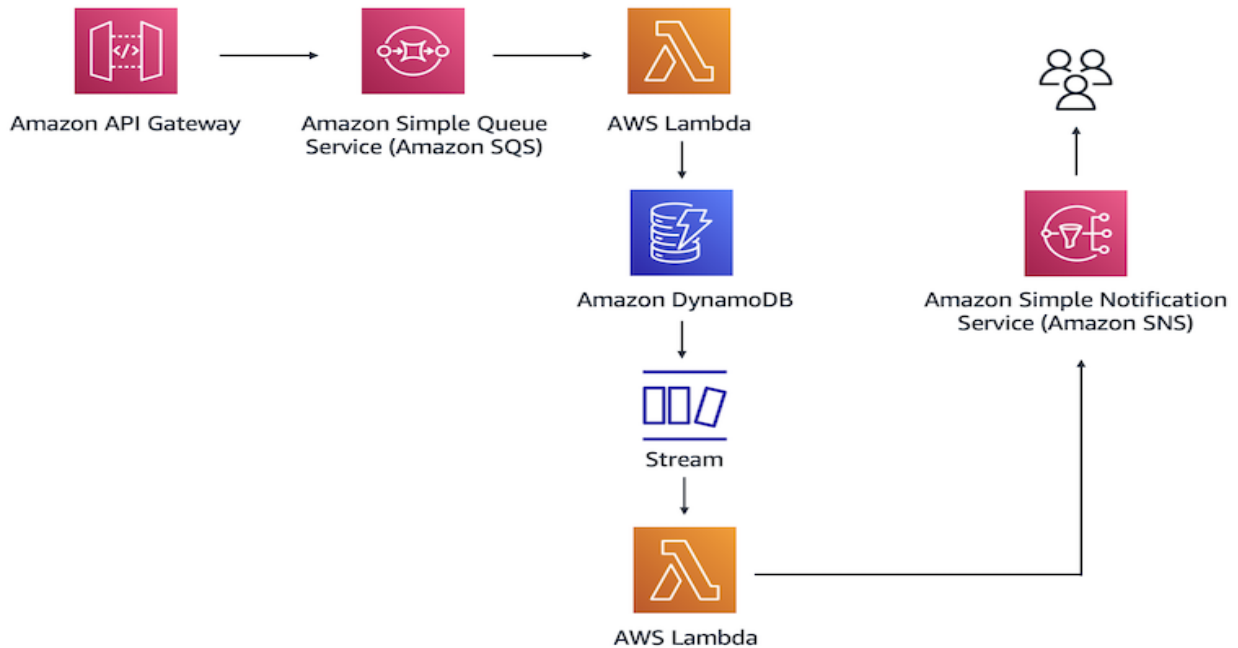
As was earlier stated, our client already has the **downstream components** migrated into the **us-east-1 region** in AWS and wants the **entire infrastructure** in this region.

ON-PREMISE MONOLITHIC ORDER SERVICE APPLICATION.



Another requirement from the client is that they want to be able to **monitor** their **application** **from one place** using **monitoring** and **observability** tools.

To achieve the above requests for our customer, we are going to follow the **workflow** specified in the **AWS architecture diagram** stated below.



1) AWS API GATEWAY AND ITS IMPORTANCE TO OUR PROJECT:

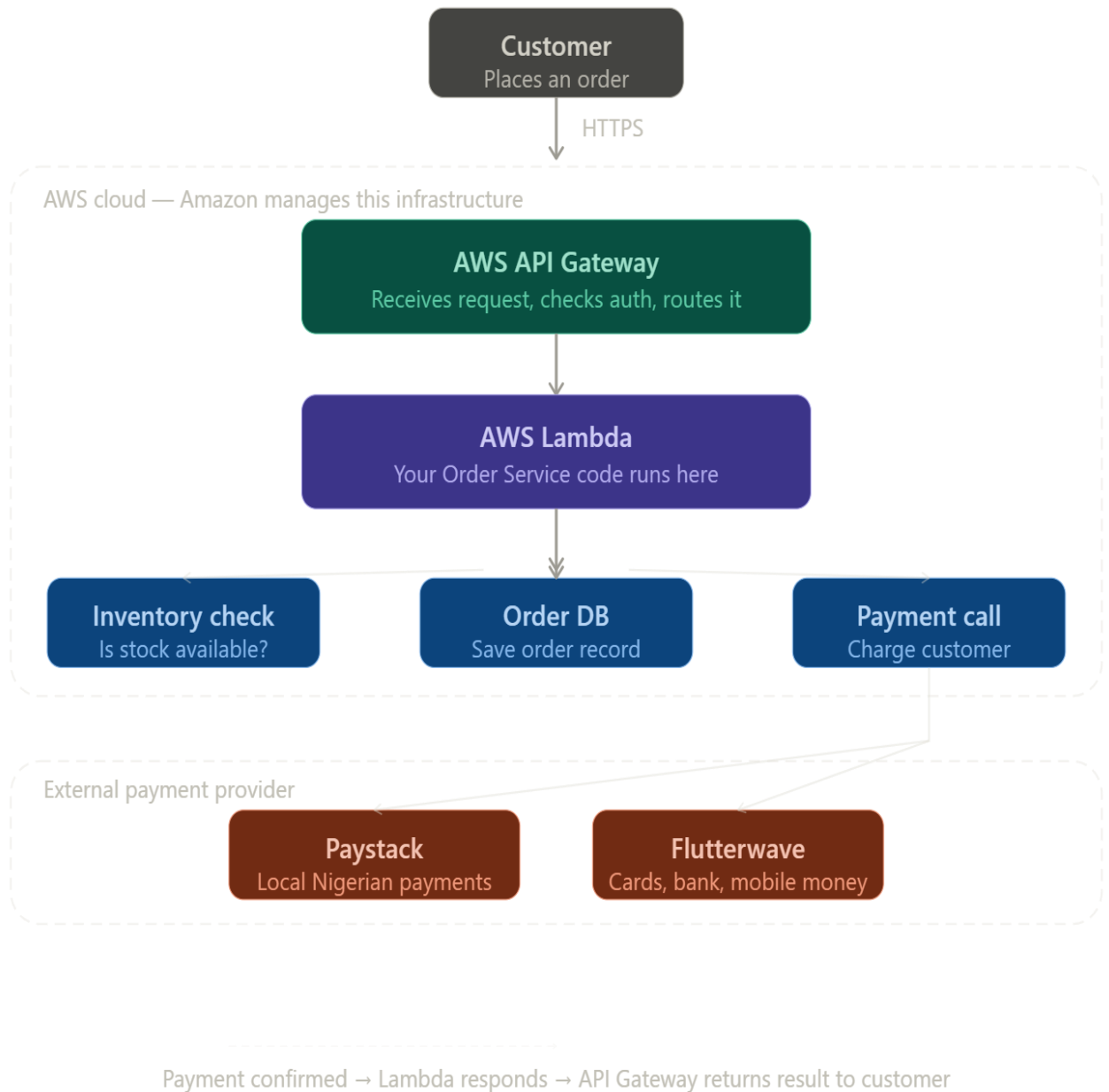
An **API Gateway** is a service gateway that will sit between our client's customer orders and the **Lambda** function that will host the order service application code, which means our client will no longer need to maintain a web server on their on-premises infrastructure to accept customer orders.

In this **architecture**, we will use an **API Gateway** from **AWS API Gateway** to authenticate our client **customer orders** requests. The **API** we will create will serve as a **communication gateway** between **customer requests** and the **AWS Lambda** function that will hosts this **order service application code** in **AWS**.

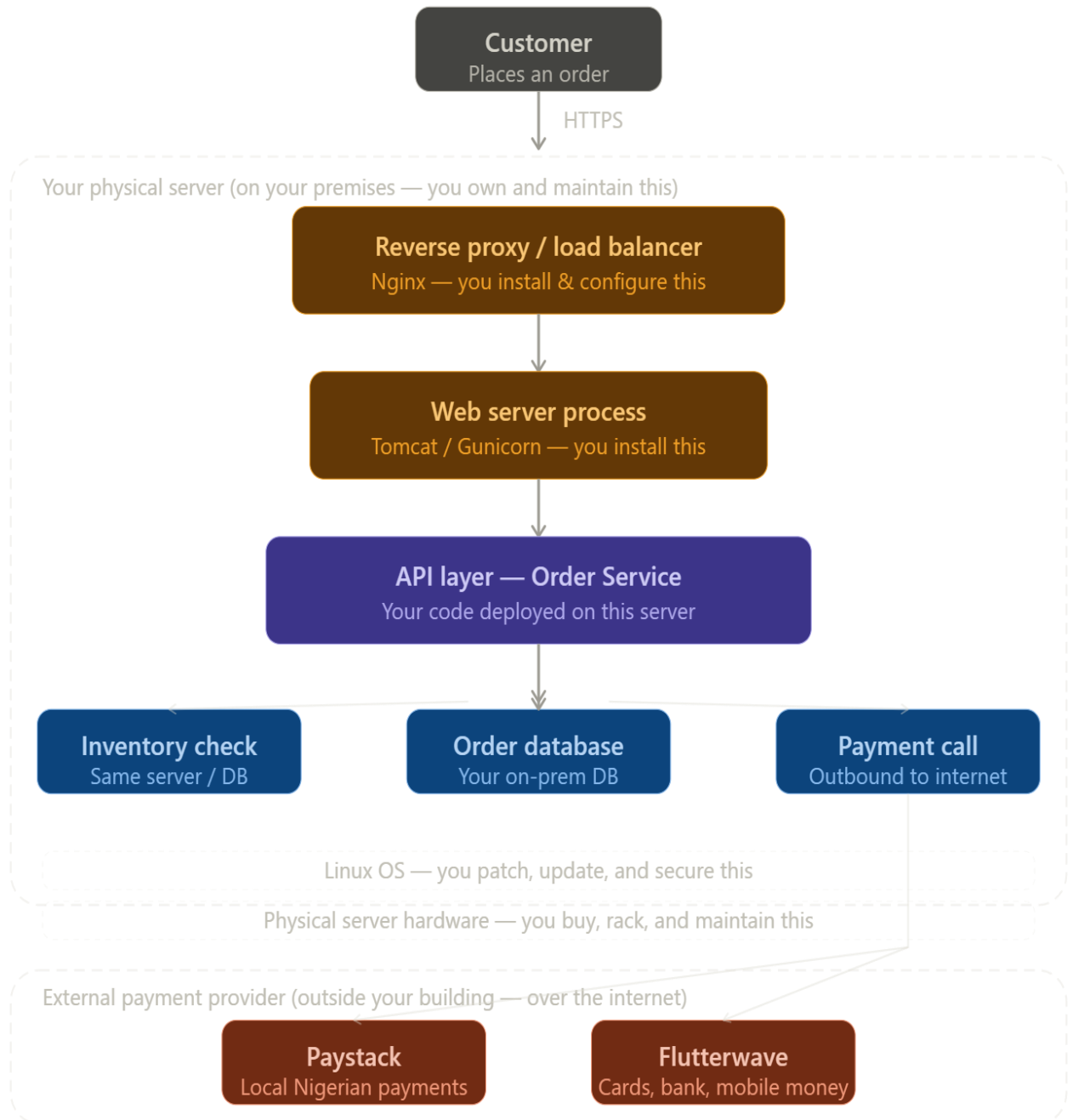
Previously, the **order service code** was being run on three (3) **Apache http (httpd)** servers on our client's **on-premises infrastructure**.

It is also important to note that the **downstream services** for our application **are already hosted on AWS**.

WITH AWS API GATEWAY ON AWS



WITHOUT AWS API GATEWAY ON-PREMISES



2) AWS LAMBDA VERSUS AWS FARGATE (ECS AND EKS) VERSUS EC2: CHOOSING THE COMPUTE SERVICE THAT FITS THE BUSINESS REQUIREMENTS.

The **Lambda function** will host the **order service application code**, but we will **rewrite** this code to **invoke** or **trigger** the **Lambda function** based on the **events** from **AWS API gateway**.

Therefore, whenever there is a **request** from a **customer** to use the **order service application** now hosted on **AWS Lambda** through a **Lambda function**, that request is authenticated by **AWS API gateway** first which triggers the **Lambda function code** to serve the **customer request**.

WHY CHOOSE AWS LAMBDA AS THE COMPUTE CHOICE.

The **compute choice** we choose to use for our client was solely determined by their **business requirements**.

First, our client requested a **serverless** and **microservice** architecture for their order service but does not have the **in-house technical skills** to manage containers like the **Elastic Container Service (ECS)** or **Elastic Kubernetes Service (EKS)**, therefore we cannot use a serverless service like **AWS fargate** for their microservice architecture.

Secondly, they specified that cost was a very big factor, and **EC2 instances** will cost our client more because they do not have a consistent demand which means that when there are no orders if we use EC2 instances, they must still run and it will cost our client more.

The best compute choice for our client is to use **AWS Lambda** which will allow our clients to pay for compute cost only when they have an order.

3) DYNAMO DB VERSUS AMAZON AURORA (AMAZON AURORA SEVERLESS): CHOOSING THE DATABASE SERVICE THAT FITS THE BUSINESS REQUIREMENTS.

Our client currently stores customer orders in an **on-premises MySQL database** service. The client wants a solution that allows them to focus on their data and not manage the **underlying heavy-lifting** that comes with **database management** like they do with their on-premises MySQL database.

To solve the problem of database management in AWS, we can choose to use a **serverless database** type for our client. The two types of **serverless database types** we can use for our clients in AWS are **DynamoDB** and **Amazon Aurora** which uses **Amazon Aurora Serverless** for **serverless needs**. Remember, **serverless** means you do not manage the underlying infrastructure of an AWS service you decide to use.

DYNAMODB: Like we already know, Amazon DynamoDB is a **NoSQL** type of database that is serverless, and can accommodate a high number of concurrent users at a go.

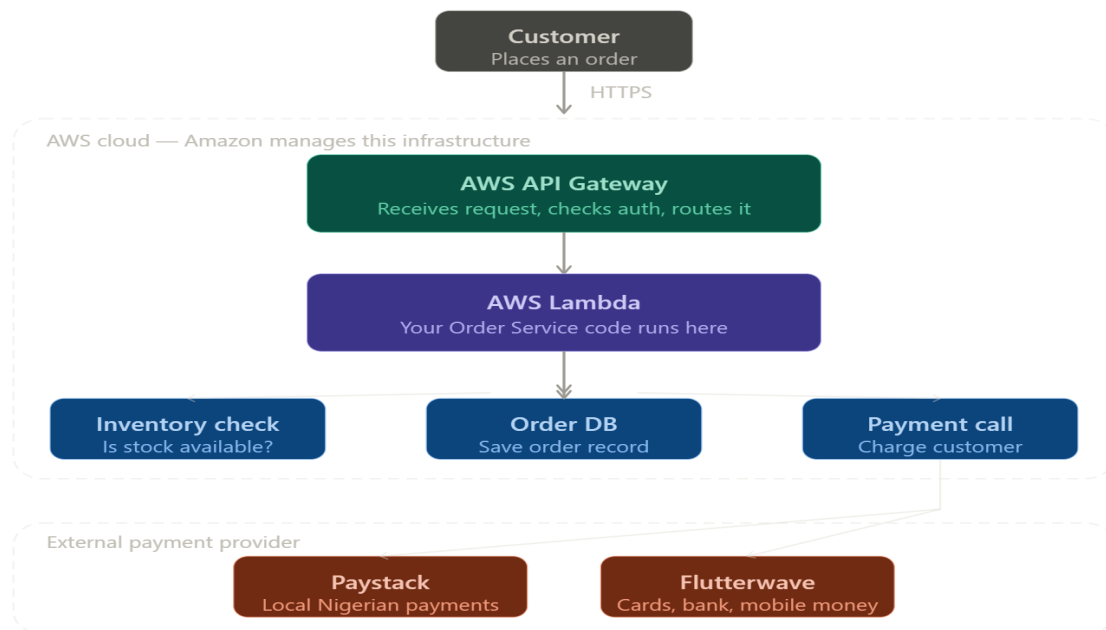
AMAZON AURORA: Amazon Aurora is a relational database that is available through Amazon RDS. It is an **SQL type** of database and can support the already existing **MySQL database service** hosted by our customer their **on-premises** infrastructure. With Amazon Aurora, our client will still manage the underlying database infrastructure but Amazon Aurora has a service called **Amazon Aurora serverless** that eliminates all of that and it uses **pay-as-go** pricing.

Our client is looking up orders by their **order ID** in a single table and won't need complex SQL queries like SQL joins, therefore we will choose to use **DynamoDB** for their database service in AWS because it is easy to use and set-up.

DynamoDB will save **customer orders** in a table.

4) AMAZON SIMPLE NOTIFICATION SERVICE (SNS) VERSUS AMAZON EVENT BRIDGE: CHOOSING THE MESSAGE SERVICE THAT FITS THE BUSINESS REQUIREMENTS.

We want to build out the part of the solution that handles the **order service users calls** to the **downstream services**.



Payment confirmed → Lambda responds → API Gateway returns result to customer

Looking at the above chart, the **downstream services** are what **allows users** of the **order service application** to **fulfil users request** like if we have **goods in our inventory**, **what they have in their carts**, or when they want to **make payments** etc.

We can **code** our **Lambda function** in **AWS lambda** to allow our **client's customers** make calls to the **downstream services** in the **order service application code**, but then those calls to the **downstream services** will still be **tightly coupled** with the **entire business logic** that makes the **order service application** which our client does not want because they don't want the failure of **one downstream service** to **affect the entire application**.

We want to **decouple** the **Order Service application code** to be **independent** of the **downstream service calls** (when users of the order service application want to check if we have goods in our inventory, what they have in their carts, or when they want to make payments etc.) and allow the **Lambda function code** to focus on taking in the **customers incoming order** from **API gateway**, **process it** and then **save it** to the **DynamoDB database table**.

When the **customer orders** are in the **DynamoDB database table**, we can now build out the solution to **capture the type of event order** (when users of the order service application want to check if we have goods in our inventory, what they have in their carts, or when they want to make payments etc.) being written to the **DynamoDB database table** and send that **event order** or **orders** to the **downstream services** (when users of the order service application want to check if we have goods in our inventory, what they have in their carts, or when they want to make payments etc.) using a **publish/subscribe model** without needing to include the **downstream services code** in the **order service application code** hosted in a **Lambda function** do it.

AMAZON SIMPLE NOTIFICATION SERVICE (SNS): Amazon simple notification service is a **serverless messaging system** that uses a **publish/subscribe (pub/sub) model** where a publisher can **send or publish messages** or **events** to something called an **SNS topic** and then you can **subscribe multiple consumers** to the topic so they receive any **published messages**.

SNS has a **one to many relationships** for **published messages** and **subscribers** where **one message** is consumed by **all subscribers** to a **topic**. SNS also supports **optional filtering** if you want **some subscribers** to **receive a message** and **not all subscribers**. Potential subscribers for **SNS topics** could be things like an **HTTPS endpoint**, **AWS lambda functions**, **Amazon SQS**, or other **AWS services**.

For **our client**, their **customer orders** would be a **message** published as an **SNS topic** to **SNS** and the **subscribers** would be the **three downstream services** these customers might want to interact with which will be in form of an **AWS service** or **AWS lambda functions**.

AMAZON EVENT BRIDGE: Amazon EventBridge is a **serverless event bus** that connects **applications using events**. This is very similar to **SNS** but there are a few differences. EventBridge supports **message filtering** by using rules which can be helpful if you have a **variety of messages** being **published** and you need to route those **specific messages** to **specific consumers**.

For our **business requirements**, all the **three downstream services** will receive **the same message** after we subscribe them to a topic in **SNS** because all the downstream services will have to fulfil our **client's customer orders in parallel**, therefore **Eventbridge** is not needed here even though it is a **very cool feature** and is a more complex and powerful tool than SNS.

Our client is empathizing on the **simplicity** and **cost** of their **infrastructure** and the **Amazon Simple Notification Service** offers that over Amazon EventBridge.

5) ADD A DYNAMO DB STREAM AND ANOTHER LAMBDA FUNCTION TO PROCESS DYNAMO DB STREAMS TO AMAZON SIMPLE NOTIFICATION SERVICE

We are going to use a **feature** called **DynamoDB streams** in **DynamoDB** to ensure that the **events (customers orders)** from the **first lambda function** are stored in our **DynamoDB** table in an **ordered flow of information** called a **stream** and can be processed by other applications.

The **DynamoDB streams** will be **processed by another application** which will be **another Lambda function**.

The **DynamoDB streams** will **trigger** the **new Lambda function** as an **event** and the **new Lambda function** will receive the **information** about the **type of customer order** added to the **DynamoDB table**. The **new Lambda function** will then take that **event** (customer order) and **submit** it as an **API call** in form of an **SNS topic** in **AWS Simple Notification Service** and the **downstream services** that will **fulfil this orders** are the **subscribers to this topic**.

This design is **handled outside** of the **order service business logic code** therefore, if there are any future changes to this current design, like adding a **new downstream service like an analytics solution** that our client customers can use to see all their purchases overtime, it is totally **decoupled** from the **order service business logic code** and does not require the **order service business logic code** to change.

6) AMAZON SIMPLE QUEUE SERVICE (SQS) VERSUS DYNAMO DB: CHOOSING THE MESSAGE SERVICE THAT FITS THE BUSINESS REQUIREMENTS.

Another **business requirement** from our client is the **latency issues** faced by its users when making **downstream calls** (when users of the order service application want to check if we have goods in our inventory, what they have in their carts, or when they want to make payments e.t.c.).

Our client is using a **synchronous (real time)** model on their **on-premises infrastructure** where **users submit their order** to the **order service application**, the **order service application accepts** and **validates** the **order**, **stores the order in a database**, then **calls the downstream services** for **inventory, fulfilment, and accounting** based on the user requests before it **responds to the user**.

This process is **slow** and **takes long to complete** which **frustrates our client users** of the **order service application**.

We will use a **storage-first pattern** to solve this issue using an **asynchronous architecture** where the request from our **client customers** will come in, **API gateway** would accept and validate the request, and then instead of sending that request to the **first Lambda function**, it would send it to a **storage service first**, this will help reduce **API latency**. Remember that our client current **AWS infrastructure design** which we are designing so far is still similar to what they have on their **on-premises infrastructure**, but we have solved the issue of **decoupling their monolithic architecture** by using **two lambda functions** in AWS Lambda and an **SNS topic** in Amazon SNS.

DYNAMO DB: If we decide to adopt the **storage-first pattern**, it means that **API Gateway** is doing the **authentication** and **data validation** of the order and **API Gateway** has a service integration with **DynamoDB**.

This makes it easy for us to get rid of the **first Lambda function** that is **accepting** and **writing** the orders to a **DynamoDB** table from AWS API Gateway. This is possible only if the **first Lambda function code** does not have any **real business logic** for the **order service application** and only the logic code to write to DynamoDB.

What it means is that if **there is real business logic code** in the **first lambda function code**, then we cannot use DynamoDB and unfortunately there is.

AMAZON SIMPLE QUEUE SERVICE: The **Amazon Simple Queue Service** is a serverless messaging queuing service that enables you to **decouple** and **scale microservices, distributed systems, and severless applications**.

This service works when **we create a queue**, and this **queue** then acts as a place to **send messages** that go between **system components**.

For our **client's infrastructure**, a message will be sent to a queue in **SQS** and stay in that queue until the **message was processed and deleted**. **SQS** works perfectly well with **Lambda**.

We will use **SQS** because the **order service application code** has some **complex business logic code** beyond **just validating and passing data** to the **DynamoDB table**.

Amazon SQS will serve as the **decoupling component** between **API Gateway** and **AWS Lambda**.

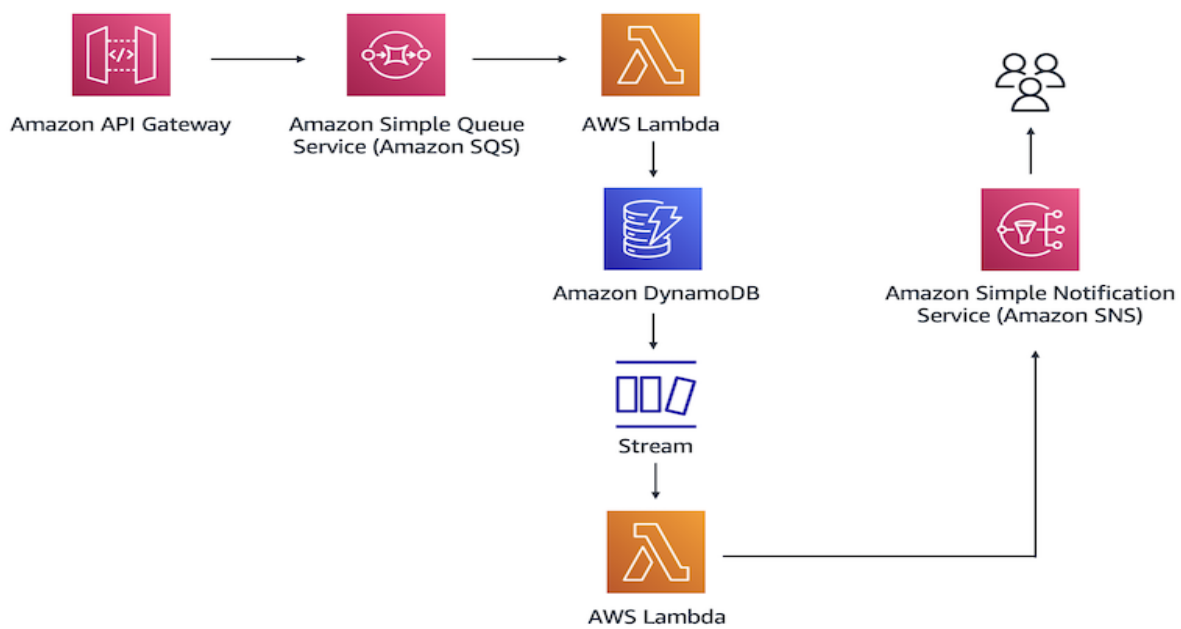
In simple terms

WITHOUT SQS:

API Gateway calls the **first Lambda function** directly and the **Lambda function** must respond immediately. if the **Lambda function** is busy, slow or fails, the customer waits which causes **latency** or **gets an error**.

WITH SQS:

API Gateway drops a **message** in a **queue** in **SQS**, and it immediately alerts the customer that there is a **message**. **Lambda** picks up the message whenever it is ready, and processes it in the background which eliminates latency.



Looking at our **architecture** above, we have an **Amazon API Gateway** that accepts customers request from the frontend of our order service application and will send a message to a **queue in SQS**, then immediately **SQS** will respond to the **user**.

From there, the **first Lambda function** in **AWS Lambda** will recognize that there is an **event** in the **queue**, run the business logic and store the information in a **DynamoDB table**.

A **DynamoDB stream** entry will then be created, which then **triggers** the **second Lambda function** in **AWS Lambda**. This **second Lambda function** will then **publish a topic** in **SNS** which publishes a message to the **downstream services** subscribed to that topic.

USING A REAL WORLD RESTAURANT (M'C DONALDS PIZZA) SCENARIO TO EXPLAIN THE ABOVE PROCESSES.

1) API GATEWAY = The receptionist at the counter

The receptionist only job is to take your order and give you a receipt. They don't cook anything. They don't know how the kitchen works. They just receive your request, make sure it's valid ("yes, we have the pizza you are requesting for"), and pass it along. The moment they hand you that receipt, their job is done and you are free to sit down.

IN YOUR INFRASTRUCTURE: API Gateway receives the customer's order from the order service application's frontend and passes it along to another service. It doesn't process anything itself, it just validates that the customer order is valid and notifies the customer immediately.

Remember that if we did not include SQS, this validation might take long to notify the customer because API gateway will have to respond **in real time** that their request is valid and if the first lambda function fails, the customer might not get a response from API gateway, so SQS solves that but not **in real time**.

2) SQS = The order screen in the kitchen

When the receptionist punches in your order, it appears on a screen in the kitchen. That screen holds your customer's order safely until someone picks it up. The receptionist doesn't wait at the screen they've already moved on to the next customer order. The order just sits there, waiting to be attended to.

IN YOUR INFRASTRUCTURE: SQS receives the message from API Gateway and holds it. It immediately confirms receipt to the customer "**your order is placed**" and waits for Lambda to pick it up. This way, the customer gets notified immediately.

3) FIRST LAMBDA FUNCTION = The kitchen worker who reads the screen

A kitchen worker sees your order appear on the screen, picks it up, and starts working. They check the ingredients, prepare the food, follow the recipe, that's the business logic. When they're done, they write everything down on the order ticket and pin it to the collection board.

IN YOUR INFRASTRUCTURE: The first Lambda function sees the SQS event, runs the **business logic code** and validates the order, calculates totals, checks inventory and saves the final record into DynamoDB.

4) DYNAMODB = The order ticket pinned to the collection board

The ticket holds all the details of the customer's completed order. It's the permanent record. But here's the key, the moment that ticket gets pinned to the board, it automatically rings a bell.

IN YOUR INFRASTRUCTURE: DynamoDB stores the order. But **DynamoDB Streams** is like that bell, the moment a new record is written, it automatically fires a notification that something changed.

5) DYNAMODB STREAM = The bell that rings when the ticket is pinned

Nobody had to manually ring that bell. It happened automatically the moment the ticket went up. That bell is what alerts the next set of people that something needs their attention.

IN YOUR INFRASTRUCTURE: The DynamoDB Stream automatically captures the fact that a new order was written, triggers the second Lambda function, and no manual wiring is needed.

6) SECOND LAMBDA FUNCTION = The manager who reads the board and makes announcements

The manager hears the bell, walks over, reads the ticket, and then picks up the intercom to announce: "**Order 47 is ready: grill station, drinks station, packaging station, your attention please.**" The manager doesn't do the grilling or the packaging. Their job is purely to broadcast the announcement.

IN YOUR INFRASTRUCTURE: The second Lambda function receives the DynamoDB Stream event, then publishes a topic to SNS, which essentially makes an announcement to the subscribed downstream services to that SNS topic.

7) SNS = The intercommunication system

The intercommunication system doesn't care who's listening. It just broadcasts the announcement restaurant-wide. Each station that needs to act on it will hear it. Stations that don't need to act on it simply ignore it.

IN YOUR INFRASTRUCTURE: SNS publishes the message to a topic. Every downstream service subscribed to that topic, like inventory, payment, notifications, etc., receives it simultaneously.

8) DOWNSTREAM SERVICES (inventory, payment, notifications, etc) = The individual stations

The grill station hears the announcement and starts grilling. The drinks station hears it and starts on the cola. The packaging station gets the bag ready. They all work in parallel, independently, without waiting for each other, and none of them needs to talk to the receptionist directly.

IN YOUR INFRASTRUCTURE: Each downstream service, as a Lambda function or AWS service, receives the SNS message and handles its own piece, like deducting stock, charging the customer, or sending a confirmation email, and they all work in parallel.

The beauty of the above design is that **the receptionist never enters the kitchen, the manager never cooks, and each station works independently**. Everything is separated by responsibility, and that is exactly what our infrastructure achieves.

WHAT REMOVING SQS MEANS FOR OUR INFRASTRUCTURE:

With SQS, API Gateway will call the first Lambda function directly and wait.

Here is what can happen if something breaks:

1) THE CUSTOMER STARES AT A LOADING SCREEN

Without SQS confirming receipt immediately, the customer has to wait for the entire order to be processed synchronously (real-time), like inventory checked, order saved, and payment

charged before they get a response. What could be a 100ms confirmation becomes a 5-10 second wait. On a slow network in Lagos, that feels broken.

2) ONE SLOW SERVICE RUINS EVERYTHING

If the payment provider, for example, Paystack, is slow to respond, Lambda is stuck waiting. API Gateway has a maximum timeout of 29 seconds. If payment takes longer than that, the entire request fails even though the order was valid and stock was available.

3) A CRASH MEANS LOST ORDERS

With SQS, if Lambda crashes mid-processing, the message is still sitting safely in the queue. Lambda retries it. Nothing is lost.

Without SQS, if Lambda crashes while processing, the order is gone. The customer got no confirmation, no error, nothing. You have no way to replay it therefore, they might need to place another order and not all customers are patient enough.

4) A TRAFFIC SPIKE TAKES EVERYTHING DOWN

During Black Friday sales, if 10,000 customers place orders simultaneously, without SQS, all 10,000 requests hit Lambda at the exact same moment. Lambda struggles to scale fast enough, requests start timing out, API Gateway starts throwing errors, and customers start seeing failures.

With SQS, all 10,000 messages land in the queue safely. Lambda processes them at a controlled pace. Every single order gets handled. The customer already got their confirmation from SQS, so they don't even notice at all.

5) EVERY SERVICE BECOMES EVERYONE ELSE'S PROBLEM

Without SQS, your order service code has to directly call inventory, payment, notifications, e.t.c. one after another. If inventory is down, you can't take orders. If payment is slow, orders back up. Every downstream problem becomes the customer's problem instantly.

In summary.

Without SQS, your entire system is only as fast, reliable, and available as your slowest and most fragile downstream service, and the customer feels every single problem in real time.

SQS is what lets you honestly tell a customer "**your order is confirmed**" the moment they place it, regardless of what happens next in the backend.

STEP 1: SIGN IN TO YOUR AWS INFRASTRUCTURE USING AN ADMIN USER.

I) CREATE A USER AND ASSIGN THAT USER **ADMINISTRATIVE PRIVILEGES**. ASSIGN THE USER THE **AdministratorAccess** POLICY, AND LOG IN WITH THAT USER USING **AN INCOGNITO WINDOW IN CHROME** TO BUILD THIS INFRASTRUCTURE.

II) I WILL USE THE USER **chinedu_2**, WHICH I CREATED IN A PREVIOUS PROJECT, TO BUILD THIS INFRASTRUCTURE. YOU CAN REFERENCE THAT PROJECT TO LEARN HOW I CREATED THE USER **chinedu_2**.

STEP 2: CREATE FOUR (4) IAM POLICIES CALLED Lambda-write-to-dynamodb, Lambda-publish-to-SNS, Lambda-read-from-dynamodb-streams, **AND** Lambda-read-from-SQS.

A) CREATE THE Lambda-write-to-dynamodb **POLICY**.

I) NAVIGATE TO **IAM** AND CLICK ON **POLICY**, AND CLICK ON **CREATE A NEW POLICY**.

II) IN THE **SPECIFY PERMISSIONS PAGE**, SWITCH TO THE **JSON TAB** AND **REPLACE THE DEFAULT PLACEHOLDER CODE** BY PASTING **THE POLICY** BELOW. ALTERNATIVELY, YOU CAN COPY THIS POLICY FROM THE **Lambda-write-to-dynamodb.json** FILE PROVIDED IN MY GITHUB REPO, THEN **CLICK NEXT**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaWriteToDynamoDB",
      "Effect": "Allow",
      "Action": [
```

```

        "dynamodb:PutItem",
        "dynamodb:DescribeTable"
    ],
    "Resource": "*"
}
]
}

```

III) GIVE YOUR **POLICY** THE NAME “**Lambda-write-to-dynamodb**”, ADD A **DESCRIPTION** “**This policy allows Lambda to write data to a dynamodb table**”, AND CLICK ON **CREATE POLICY**.

IV) THE **Lambda-write-to-dynamodb POLICY** WILL **GRANT PERMISSIONS FOR WRITE RECORDS TO A DYNAMODB TABLE**.

B) CREATE THE Lambda-publish-to-SNS POLICY.

I) NAVIGATE TO **IAM** AND CLICK ON **POLICY**, AND CLICK ON **CREATE A NEW POLICY**.

II) IN THE **SPECIFY PERMISSIONS PAGE**, SWITCH TO THE **JSON TAB** AND **REPLACE THE DEFAULT PLACEHOLDER CODE** BY PASTING **THE POLICY** BELOW. ALTERNATIVELY, YOU CAN COPY THIS POLICY FROM THE **Lambda-publish-to-SNS.json** FILE PROVIDED IN MY GITHUB REPO, THEN **CLICK NEXT**.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaPublishTopicsToSNS",
      "Effect": "Allow",
      "Action": [
        "sns:Publish",
        "sns:GetTopicAttributes",
        "sns:ListTopics"
      ],
      "Resource": "*"
    }
  ]
}

```

III) GIVE YOUR **POLICY** THE NAME “**Lambda-publish-to-SNS**”, ADD A **DESCRIPTION** “**This policy allows Lambda to create an SNS topic**”, AND CLICK ON **CREATE POLICY**.

IV) THE **Lambda-publish-to-SNS POLICY** WILL GRANT PERMISSIONS FOR LAMBDA TO CREATE A TOPIC IN AMAZON SIMPLE NOTIFICATION SERVICE (SNS).

C) CREATE THE Lambda-read-from-dynamodb-streams POLICY.

I) NAVIGATE TO **IAM** AND CLICK ON **POLICY**, AND CLICK ON **CREATE A NEW POLICY**.

II) IN THE **SPECIFY PERMISSIONS PAGE**, SWITCH TO THE **JSON TAB** AND **REPLACE THE DEFAULT PLACEHOLDER CODE** BY PASTING THE **POLICY** BELOW. ALTERNATIVELY, YOU CAN COPY THIS POLICY FROM THE **Lambda-read-from-dynamodb-streams.json** FILE PROVIDED IN MY GITHUB REPO, THEN **CLICK NEXT**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaReadinDynamoDBStreams",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetShardIterator",
        "dynamodb:DescribeStream",
        "dynamodb:ListStreams",
        "dynamodb:GetRecords"
      ],
      "Resource": "*"
    }
  ]
}
```

III) GIVE YOUR **POLICY** THE NAME **"Lambda-read-from-dynamodb-streams"**, ADD A **DESCRIPTION** **"This policy allows Lambda to read data inside dynamodb streams"**, AND **CLICK ON CREATE POLICY**.

IV) THE **Lambda-read-from-dynamodb-streams POLICY** WILL ALLOW LAMBDA TO READ DATA INSIDE DYNAMODB STREAMS.

D) CREATE THE Lambda-read-from-SQS POLICY.

I) NAVIGATE TO **IAM** AND CLICK ON **POLICY**, AND CLICK ON **CREATE A NEW POLICY**.

II) IN THE **SPECIFY PERMISSIONS PAGE**, SWITCH TO THE **JSON TAB** AND **REPLACE THE DEFAULT PLACEHOLDER CODE** BY PASTING **THE POLICY** BELOW. ALTERNATIVELY, YOU CAN COPY THIS POLICY FROM THE **Lambda-read-from-SQS.json** FILE PROVIDED IN MY GITHUB REPO, THEN **CLICK NEXT**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaReadFromSQS",
      "Effect": "Allow",
      "Action": [
        "sqs:DeleteMessage",
        "sqs:ReceiveMessage",
        "sqs:GetQueueAttributes",
        "sqs:ChangeMessageVisibility"
      ],
      "Resource": "*"
    }
  ]
}
```

III) GIVE YOUR **POLICY** THE NAME **“Lambda-read-from-SQS”**, ADD A **DESCRIPTION “This policy allows Lambda to read orders from SQS”**, AND **CLICK ON CREATE POLICY**.

IV) THE **Lambda-read-from-SQS POLICY** WILL **ALLOW LAMBDA TO READ INCOMING CUSTOMERS' ORDERS FROM THE SQS QUEUE**.

Search for all the **created policies** and **inspect them**. Observe that they are **customer-managed policies** because we created them.

STEP 3: CREATE THREE (3) IAM ROLES CALLED **Lambda-read-from-SQS-write-to-dynamodb**, **lambda-read-from-dynamodbstreams-write-to-SNS**, **AND** **ApiGateway-write-to-SQS-CloudWatchlogs** **ROLES AND ATTACH THE CREATED POLICIES WHERE DUE INSIDE THESE ROLES**.

A) CREATE THE Lambda-read-from-SQS-send-to-dynamodb ROLE.

I) NAVIGATE TO **IAM** AND CLICK ON **ROLE**, AND CLICK ON **CREATE A NEW ROLE**.

II) UNDER **TRUSTED ENTITY TYPE**, SELECT **AWS SERVICE**, AND UNDER USE CASE SELECT **LAMBDA** AND CLICK ON **NEXT** AND CLICK ON **NEXT AGAIN**.

III) **IN THE ADD PERMISSIONS PAGE, SEARCH FOR THE lambda-read-from-SQS POLICY AND THE Lambda-write-to-dynamodb POLICY WE CREATED IN STEP 2, CLICK ON THEIR CHECK BOXES, AND CLICK ON NEXT.**

IV) GIVE YOUR **ROLE THE NAME** “Lambda-read-from-SQS-write-to-dynamodb” REVIEW YOUR CONFIGURATIONS AND CLICK ON **CREATE ROLE**.

V) COPY THE **ARN OF THIS ROLE** IN A SEPARATE SHEET. WE WILL NEED IT IN THE FUTURE. IT SHOULD LOOK LIKE THE BELOW.

arn:aws:iam::140023390772:role/Lambda-read-from-SQS-write-to-dynamodb

B) CREATE THE lambda-read-from-dynamodbstreams-write-to-SNS ROLE.

I) NAVIGATE TO **IAM** AND CLICK ON **ROLE**, AND CLICK ON **CREATE A NEW ROLE**.

II) UNDER **TRUSTED ENTITY TYPE**, SELECT **AWS SERVICE**, AND UNDER USE CASE SELECT **LAMBDA** AND CLICK ON **NEXT** AND CLICK ON **NEXT AGAIN**.

III) **IN THE ADD PERMISSIONS PAGE, SEARCH FOR THE Lambda-read-from-dynamodb-streams POLICY AND THE Lambda-publish-to-SNS POLICY WE CREATED IN STEP 2, CLICK ON THEIR CHECK BOXES, AND CLICK ON NEXT.**

IV) GIVE YOUR **ROLE THE NAME** “lambda-read-from-dynamodbstreams-write-to-SNS” REVIEW YOUR CONFIGURATIONS AND CLICK ON **CREATE ROLE**.

V) COPY THE **ARN OF THIS ROLE** IN A SEPARATE SHEET. WE WILL NEED IT IN THE FUTURE. IT SHOULD LOOK LIKE THE BELOW.

arn:aws:iam::140023390772:role/lambda-read-from-dynamodbstreams-write-to-SNS

C) CREATE THE ApiGateway-push-to-SQS-CloudWatchlogs ROLE TO PUSH CUSTOMERS' REQUESTS TO AN SQS QUEUE AND AMAZON CLOUDWATCH LOGS FOR TROUBLESHOOTING ERRORS.

I) NAVIGATE TO **IAM** AND CLICK ON **ROLE**, AND CLICK ON **CREATE A NEW ROLE**.

II) UNDER **TRUSTED ENTITY TYPE**, SELECT **AWS SERVICE**, AND UNDER USE CASE SELECT **API GATEWAY** AND CLICK ON **NEXT** AND CLICK ON **NEXT AGAIN**.

III) IN THE **ADD PERMISSIONS PAGE**, THE **AWS MANAGED POLICY AmazonAPIGatewayPushToCloudWatchLogs** IS **ALREADY ATTACHED**. **CLICK ON NEXT**.

IV) GIVE YOUR **ROLE THE NAME** "ApiGateway-push-to-SQS-CloudWatchlogs". REVIEW YOUR CONFIGURATIONS AND CLICK ON **CREATE ROLE**.

V) COPY THE **ARN OF THIS ROLE** IN A SEPARATE SHEET. WE WILL NEED IT IN THE FUTURE. IT SHOULD LOOK LIKE THE BELOW.

arn:aws:iam::140023390772:role/ApiGateway-write-to-SQS-CloudWatchlogs

INSPECT ALL THE CREATED ROLES AND OBSERVE YOUR ATTACHED POLICIES TO THOSE ROLES.

STEP 4: CREATE AN SQS QUEUE TO RECEIVE ORDER RECORDS FROM API GATEWAY.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **SQS** AND IN THE SQS DASHBOARD, CLICK ON **CREATE QUEUE**.

III) IN THE **CREATE QUEUE PAGE**, UNDER DETAILS, SELECT **STANDARD**, AND UNDER NAME, YOU CAN GIVE YOUR QUEUE ANY NAME. FOR ME, I WILL NAME MY QUEUE **orders_queue**.

IV) LOCATE ENCRYPTION, UNDER **SERVER-SIDE ENCRYPTION**, SELECT **ENABLED**, AND UNDER **ENCRYPTION KEY TYPE**, SELECT **Amazon SQS key (SSE-SQS)**.

V) LOCATE **ACCESS POLICY**, UNDER CHOOSE METHOD SELECT **BASIC**, AND UNDER **DEFINE WHO CAN SEND MESSAGES TO THE QUEUE**, SELECT **Only the specified AWS accounts, IAM users and roles**, AND PASTE THE **ARN OF THE ApiGateway-write-to-SQS-CloudWatchlogs** ROLE WE CREATED. IT SHOULD LOOK LIKE `arn:aws:iam::140023390772:role/ApiGateway-write-to-SQS-CloudWatchlogs`.

VI) UNDER **DEFINE WHO CAN RECEIVE MESSAGES FROM THE QUEUE**, SELECT **Only the specified AWS accounts, IAM users and roles**, AND PASTE THE ARN OF THE **Lambda-read-from-SQS-write-to-dynamodb** ROLE WE CREATED. IT SHOULD LOOK LIKE `arn:aws:iam::140023390772:role/Lambda-read-from-SQS-write-to-dynamodb`.

VII) LEAVE THE OTHER DEFAULT OPTIONS AND CLICK ON **CREATE QUEUE**.

VIII) AFTER **CREATING YOUR QUEUE**, **COPY THE ARN** OF YOUR QUEUE, AND **PASTE IT** IN A SEPARATE SHEET. WE ARE GOING TO USE IT IN THE FUTURE.

YOUR ARN MIGHT LOOK LIKE `arn:aws:sqs:us-east-1:140023390772:orders_queue`

STEP 5: CREATE A DYNAMODB TABLE TO INGEST AND STORE ORDERS FROM AWS LAMBDA.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **DYNAMO DB** AND IN THE DYNAMO DB DASHBOARD, CLICK ON **CREATE TABLE**.

III) IN THE **CREATE TABLE PAGE**, UNDER **TABLE NAME**, GIVE YOUR TABLE THE NAME **orders**, AND UNDER **PARTITION KEY**, NAME IT **orderId**, AND LEAVE IT AT THE **STRING DATA TYPE**. LEAVE THE DEFAULT SETTINGS AND CLICK ON **CREATE TABLE**.

STEP 6: CREATE A LAMBDA FUNCTION THAT READS ORDERS FROM SQS AND STORES THOSE ORDERS IN A DYNAMO DB TABLE.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **AWS LAMBDA**, AND IN THE **LAMBDA DASHBOARD**, CLICK ON **CREATE A FUNCTION**.

III) IN THE **CREATE FUNCTION PAGE**, SELECT USE **AUTHOR FROM SCRATCH**.

IV) UNDER **BASIC INFORMATION**, LOCATE **FUNCTION NAME**, GIVE YOUR FUNCTION ANY NAME. IN MY CASE, I WILL GIVE MY FUNCTION THE NAME **read-SQS-and-store-in-dynamodb**

UNDER **RUNTIME**, SELECT **PYTHON 3.14**. NOTE THAT THE **PYTHON VERSION** USED IS THE **LATEST PYTHON VERSION RELEASE** AND MIGHT CHANGE TO THE **LATEST** WHEN YOU ARE **ATTEMPTING THIS PROJECT**.

V) UNDER **ADDITIONAL SETTINGS**, LOCATE **CONFIGURE CUSTOM EXECUTION ROLE** AND **ENABLE IT**, UNDER **EXECUTION ROLE**, SELECT **CHOOSE EXISTING ROLE**. CLICK ON THE **DROP-DOWN** AND SELECT THE **Lambda-read-from-SQS-write-to-dynamodb** ROLE WE HAD CREATED INITIALLY IN **STEP 3**. LEAVE THE OTHER DEFAULTS AND CLICK ON **CREATE FUNCTION**.

CREATE AN SQS TRIGGER TO INVOKE THE LAMBDA FUNCTION.

VI) IN THE **read-SQS-and-store-in-dynamodb** LAMBDA FUNCTION PAGE WE HAVE CREATED, UNDER **FUNCTION OVERVIEW**, CLICK ON THE **+ ADD TRIGGER** BUTTON. IN THE **ADD TRIGGER** PAGE, SEARCH FOR **SQS**, SELECT IT, UNDER **SQS QUEUE**, SELECT THE **orders_queue** WE CREATED, LEAVE THE OTHER DEFAULTS, AND CLICK ON **ADD**.

ADD THE LAMBDA FUNCTION CODE.

VII) BACK TO THE **read-SQS-and-store-in-dynamodb** LAMBDA FUNCTION PAGE WE CREATED, SCROLL DOWN, LOCATE THE **CODE TAB**, AND **CLICK ON IT**.

VIII) UNDER **CODE SOURCE**, AND UNDER **EXPLORER**, IN YOUR **lambda_function.py** FILE, REPLACE THE DEFAULT PYTHON SCRIPT WITH THE ONE BELOW, OR USE THE **send_sqs_orders_to_dynamodb.py** PYTHON FILE.

```
import boto3
import uuid

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table("orders")

def lambda_handler(event, context):
    for record in event['Records']:
        payload = record["body"]
        print(f"Processing record: {payload}")
        table.put_item(
            Item={
                'orderId': str(uuid.uuid4()),
                'order': payload
            }
        )
```

The sole job of the **Python script above** is to ensure that when customers purchase an order, it is saved as an SQS queue, and that queue, which is an event, triggers a lambda function. This lambda function will send those customers orders to the orders DynamoDB table and record the orders (payload) in that table.

IX) LOCATE **DEPLOY** UNDER **EXPLORER** AND CLICK ON **DEPLOY**, AND YOU SHOULD SEE THE MESSAGE **SUCCESSFULLY UPDATED THE FUNCTION read-SQS-and-store-in-dynamodb**.

X) WHENEVER THERE IS AN **EVENT (orders)** FROM **SQS**, IT TRIGGERS THE **LAMBDA FUNCTION CODE** TO PUSH THAT **EVENT** TO THE **orders DYNAMO DB** TABLE WE CREATED.

TESTING THE read-SQS-and-store-in-dynamodb LAMBDA FUNCTION.

XI) LOCATE **EXPLORER**, AND UNDER **TEST EVENTS [NONE SELECTED]**, CLICK ON **CREATE NEW TEST EVENT**.

XII) IN THE **CREATE NEW TEST EVENT** PAGE, GIVE YOUR TEST EVENT ANY NAME. FOR ME, I WILL USE THE NAME **SQS-lambda-test-event**

XIII) SCROLL DOWN, AND UNDER **TEMPLATE_OPTIONAL**, SELECT **SQS**. YOU SHOULD SEE A **TEMPLATE SQS JSON SCRIPT**. CLICK ON **SAVE**.

XIV) LOCATE **EXPLORER**, AND CLICK ON **TEST**, AND YOU SHOULD SEE THE STATUS OF YOUR TEST **SUCCESSING** IF ALL YOUR CONFIGURATIONS ARE CORRECT IN THE **OUTPUT**.

VERIFY THAT THE LAMBDA FUNCTION ADDS THE TEST MESSAGE TO THE DYNAMO DB TABLE.

XV) TO CONFIRM THAT THE **TEST RESULTS** ARE INPUT INTO THE **orders** DYNAMO DB TABLE, LOCATE YOUR DYNAMO DB TABLE AND CLICK ON THE **orders** TABLE YOU CREATED, CLICK ON **EXPLORE TABLE ITEMS**, SCROLL DOWN, AND YOU WILL SEE THE TEST RESULTS FROM OUR **LAMBDA FUNCTION** IN THE **orders** DYNAMO DB TABLE.

STEP 7: ENABLE DYNAMO DB STREAMS TO TRIGGER THE SECOND LAMBDA FUNCTION.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **DYNAMO DB** IN THE SEARCH BAR AND CLICK ON THE **orders** TABLE YOU CREATED.

III) CLICK ON THE **SCROLL RIGHT BUTTON >**, LOCATE THE **EXPORTS AND STREAMS** TAB, AND CLICK ON IT.

IV) SCROLL DOWN TO **DYNAMODB STREAMS DETAILS** AND CLICK ON **TURN ON**.

V) IN THE **DYNAMODB STREAMS DETAILS** UNDER VIEW TYPE, SELECT **NEW IMAGE** AND CLICK ON **TURN ON STREAM**.

THIS WILL ENABLE DYNAMO DB STREAM ON THE orders TABLE.

STEP 8: CREATE AN SNS TOPIC AND ENABLE THE DOWNSTREAM SERVICES TO SUBSCRIBE TO THAT TOPIC.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **SNS** IN THE SEARCH BAR AND IN THE **SNS DASHBOARD**, CLICK ON **NEXT STEP**.

III) IN THE **CREATE TOPIC PAGE**, UNDER **DETAILS**, SELECT **STANDARD**, AND UNDER NAME, YOU CAN GIVE YOUR QUEUE ANY NAME. FOR ME, I WILL NAME MY QUEUE **sns_orders_topic**.

IV) LOCATE **ACCESS POLICY**, UNDER **CHOOSE METHOD** SELECT **ADVANCED**. IN THE JSON SCRIPT, LOCATE **AWS**, REMOVE THE *, AND PASTE THE **lambda-read-from-dynamodbstreams-write-to-SNS** ROLE'S ARN BECAUSE WE WANT **DYNAMODB STREAMS** TO **PUBLISH/WRITE** TO **SNS**. THE ARN SHOULD LOOK LIKE **arn:aws:iam::140023390772:role/lambda-read-from-dynamodbstreams-write-to-SNS**. LEAVE THE **OTHER DEFAULTS** AND CLICK ON **CREATE TOPIC**.

V) COPY THE **ARN** OF YOUR **SNS TOPIC**. YOU WILL NEED IT LATER IN THE FUTURE.

VI) CLICK ON YOUR **SNS TOPIC**, CLICK ON THE **SUBSCRIPTIONS TAB**, AND CLICK ON **CREATE SUBSCRIPTION**.

VII) IN THE **CREATE SUBSCRIPTION PAGE**, UNDER **PROTOCOL**, SELECT **EMAIL**, AND UNDER **ENDPOINT**, INPUT YOUR OWN **EMAIL ADDRESS**. LEAVE THE **OTHER DEFAULTS** AND CLICK ON **CREATE SUBSCRIPTION**, AND A **SUBSCRIPTION EMAIL** WILL BE SENT TO YOU.

VIII) OPEN THE **SENT EMAIL** AND CLICK ON **CONFIRM SUBSCRIPTION**. YOU SHOULD SEE A **SUBSCRIPTION CONFIRMED MESSAGE FROM AWS**.

STEP 9: CREATE A LAMBDA FUNCTION THAT READS ORDERS FROM DYNAMO DB STREAMS AND PUBLISHES A MESSAGE TO AN SNS TOPIC.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **AWS LAMBDA**, AND IN THE **LAMBDA DASHBOARD**, CLICK ON **CREATE A FUNCTION**.

III) IN THE **CREATE FUNCTION PAGE**, SELECT USE **AUTHOR FROM SCRATCH**.

IV) UNDER **BASIC INFORMATION**, LOCATE **FUNCTION NAME**, GIVE YOUR FUNCTION ANY NAME. IN MY CASE, I WILL GIVE MY FUNCTION THE NAME **read-dynamodbstreams-and-write-to-sns-topic**

UNDER **RUNTIME**, SELECT **PYTHON 3.14**. NOTE THAT THE **PYTHON VERSION** USED IS THE **LATEST PYTHON VERSION RELEASE** AND MIGHT CHANGE TO THE **LATEST** WHEN YOU ARE **ATTEMPTING THIS PROJECT**.

V) UNDER **ADDITIONAL SETTINGS**, LOCATE **CONFIGURE CUSTOM EXECUTION ROLE** AND **ENABLE IT**, UNDER **EXECUTION ROLE**, SELECT **CHOOSE EXISTING ROLE**. CLICK ON THE **DROP-DOWN** AND SELECT THE **Lambda-read-from-dynamodbstreams-write-to-SNS** ROLE WE HAD CREATED INITIALLY IN **STEP 3**. LEAVE THE OTHER DEFAULTS AND CLICK ON **CREATE FUNCTION**

CREATE A DYNAMODB TRIGGER TO INVOKE THE LAMBDA FUNCTION.

VI) IN THE NAME **read-dynamodbstreams-and-write-to-sns-topic** LAMBDA FUNCTION PAGE WE HAVE CREATED, UNDER **FUNCTION OVERVIEW**, CLICK ON THE **+ ADD TRIGGER** BUTTON. IN THE **ADD TRIGGER** PAGE, SEARCH FOR **DYNAMODB**, SELECT IT, UNDER **DYNAMODB TABLE**, SELECT THE **orders** TABLE WE CREATED, LEAVE THE OTHER DEFAULTS, AND CLICK ON **ADD**.

ADD THE LAMBDA FUNCTION CODE.

VII) BACK TO THE **read-dynamodbstreams-and-write-to-sns-topic** LAMBDA FUNCTION PAGE WE CREATED, SCROLL DOWN, LOCATE THE **CODE TAB**, AND **CLICK ON IT**.

VIII) UNDER **CODE SOURCE**, AND UNDER **EXPLORER**, IN YOUR **lambda_function.py** FILE, REPLACE THE DEFAULT PYTHON SCRIPT WITH THE ONE BELOW, OR USE THE **dynamodbstreams_to_sns_topic.py** PYTHON FILE. **REPLACE THE TARGET ARN WITH THE ARN OF YOUR SNS TOPIC IN THE SCRIPT BELOW**.

```
import boto3
import json

# Connect to SNS using the AWS SDK
client = boto3.client('sns')

def lambda_handler(event, context):

    # DynamoDB Streams can deliver multiple record changes at once, loop through
    # each
    for record in event["Records"]:

        # Only act on new records being inserted – ignore updates and deletes
```

```

if record['eventName'] == 'INSERT':

    # Extract the newly inserted item from the DynamoDB stream record
    new_record = record['dynamodb']['NewImage']

    # Publish the new order as a message to the SNS topic
    response = client.publish(
        TargetArn='<Enter Amazon SNS ARN for your sns-topic>', # the SNS
topic to broadcast to

        # Wrap the record in JSON format – outer layer tells SNS the
structure, inner layer is the actual data
        Message=json.dumps({'default': json.dumps(new_record)}),

        # Tells SNS the message is formatted differently per channel
(email, SQS, Lambda etc.)
        MessageStructure='json'
    )

```

The sole job of the **Python script above** is to ensure that only new orders are sent to the SNS topic and that the message being sent is in a JSON format.

IX) LOCATE **DEPLOY** UNDER **EXPLORER** AND CLICK ON **DEPLOY**, AND YOU SHOULD SEE THE MESSAGE **SUCCESSFULLY UPDATED THE FUNCTION read-dynamodbstreams-and-write-to-sns-topic**.

X) WHENEVER THERE IS AN **EVENT (orders)** FROM **DYNAMODB STREAMS**, IT TRIGGERS THE **LAMBDA FUNCTION CODE** TO SEND THAT **EVENT** TO THE **sns_orders_topic** WE CREATED IN SNS.

TESTING THE read-dynamodbstreams-and-write-to-sns-topic LAMBDA FUNCTION.

XI) LOCATE **EXPLORER**, AND UNDER **TEST EVENTS [NONE SELECTED]**, CLICK ON **CREATE NEW TEST EVENT**.

XII) IN THE **CREATE NEW TEST EVENT** PAGE, GIVE YOUR TEST EVENT ANY NAME. FOR ME, I WILL USE THE NAME **dynamo-lambda-test-event**

XIII) SCROLL DOWN, AND UNDER **TEMPLATE_OPTIONAL**, SELECT **DYNAMODB UPDATE**. YOU SHOULD SEE A **TEMPLATE DYNAMODB JSON SCRIPT**. CLICK ON **SAVE**.

XIV) LOCATE **EXPLORER**, AND CLICK ON **TEST**, AND YOU SHOULD SEE THE STATUS OF YOUR TEST **SUCCESSING** IF ALL YOUR CONFIGURATIONS ARE CORRECT IN THE **OUTPUT**.

XV) **CHECK YOUR EMAIL** TO SEE THAT YOU **RECEIVED AN EMAIL FROM SNS**.

STEP 10: CREATE AN API GATEWAY TO ALLOW THE ORDER SERVICE APPLICATION TO COMMUNICATE WITH YOUR AWS SERVICES.

I) MAKE SURE YOU ARE LOGGED IN AS AN **ADMIN USER**. SELECT THE **us-east-1** REGION.

II) SEARCH FOR **API GATEWAY**, AND IN THE **API DASHBOARD**, CLICK ON **CREATE AN API**.

III) IN THE **CHOOSE AN API TYPE PAGE**, SELECT **REST API** BY CLICKING **BUILD**.

IV) IN THE **CREATE REST API PAGE**, UNDER API DETAILS, SELECT **NEW API**.

V) UNDER **API NAME**, GIVE YOUR API ANY NAME. FOR ME, I WILL GIVE MY API THE NAME **orders-service-SQS-api**, AND UNDER **DESCRIPTION (OPTIONAL)**, INPUT **"This api will allow customer orders to be placed in a queue until it is attended to"** LEAVE THE OTHER DEFAULTS, AND CLICK ON **CREATE API**.

VI) IN THE **RESOURCES PAGE**, CLICK ON **CREATE RESOURCE** AND UNDER **RESOURCE NAME**, GIVE YOUR RESOURCE THE NAME **order-requests**, AND CLICK ON **CREATE RESOURCE**.

VII) IN THE **RESOURCES PAGE**, UNDER **METHODS**, CLICK ON **CREATE METHOD**.

VIII) IN THE **CREATE METHOD PAGE**, UNDER **METHOD TYPE**, SELECT **POST**.

UNDER **INTEGRATION TYPE**, SELECT **AWS SERVICE**.

UNDER **AWS REGION**, SELECT **us-east-1**.

UNDER **AWS SERVICE**, SELECT **Simple Queue Service (SQS)**.

UNDER **AWS SUBDOMAIN**, LEAVE **EMPTY**.

UNDER **HTTP METHOD**, SELECT **POST** BECAUSE WE WANT THIS API TO POST TO **SQS**.

UNDER **ACTION TYPE**, CHOOSE **USE PATH OVERRIDE**, AND IN THE **PATH OVERRIDE** TAB, TYPE IN YOUR AWS ACCOUNT ID, ADD A BACKSLASH, AND INPUT THE NAME OF YOUR SQS QUEUE. IT SHOULD LOOK LIKE **140023390772/orders_queue**

UNDER **EXECUTION ROLE**, PASTE THE **ARN** OF THE **ApiGateway-write-to-SQS-CloudWatchlogs** **ROLE** WE CREATED INITIALLY. IT SHOULD LOOK LIKE **arn:aws:iam::140023390772:role/ApiGateway-write-to-SQS-CloudWatchlogs**, LEAVE THE **REMAINING DEFAULT CONFIGURATIONS** AND CLICK ON **CREATE METHOD**.

IX) IN THE **RESOURCES DASHBOARD**, LOCATE THE **INTEGRATION REQUEST TAB** AND CLICK ON **EDIT**.

IN THE **EDIT INTEGRATION REQUEST PAGE**, **SCROLL DOWN**, LOCATE **REQUEST BODY PASSTHROUGH**, AND CLICK ON **NEVER**.

SCROLL DOWN AND LOCATE **URL REQUEST HEADERS PARAMETERS** AND CLICK ON THE **DROP-DOWN**.

UNDER **NAME**, TYPE **Content-Type**, AND UNDER **MAPPED FROM**, TYPE **'application/x-www-form-urlencoded'**

LOCATE **MAPPING TEMPLATES**, CLICK ON THE **DROP-DOWN**, AND CLICK ON **ADD MAPPING TEMPLATE**. UNDER **CONTENT TYPE**, TYPE IN **application/json**, UNDER **GENERATE TEMPLATE** TAB, LEAVE IT **EMPTY**, AND IN THE **TEMPLATE BODY**, COPY AND PASTE THE **VELOCITY TEMPLATE LANGUAGE SCRIPT** BELOW AND CLICK **SAVE**.

```
Action=SendMessage&MessageBody=${input.body}
```

NOTE THAT THE **VELOCITY TEMPLATE LANGUAGE SCRIPT** WILL VALIDATE **CUSTOMERS' ORDER REQUESTS** FROM THE **API GATEWAY** TO WHAT **SQS UNDERSTANDS**.

X) WE WANT TO **TEST THE EVENTS OF CUSTOMER ORDERS** FROM **API GATEWAY**.

IN THE **RESOURCE PAGE**, **SCROLL DOWN** AND BY YOUR **FAR RIGHT LOCATE THE TEST TAB** AND **CLICK ON IT**. IN THE **REQUEST BODY**, **COPY AND PASTE THE JSON SCRIPT BELOW** AND **CLICK ON TEST**.

```
{"item": "noise canceling headset",  
"customerID": "12345"}
```

REVIEW THE BODY OF THE **REQUEST LOGS**, AND WE SHOULD SEE THE MESSAGE: **status 200** AT THE TOP, WHICH MEANS **API GATEWAY PROCESSED OUR REQUEST SUCCESSFULLY**.

ALSO IN THE MIDDLE OF THE **REQUEST LOGS**, YOU SHOULD SEE THE BELOW:

Tue Apr 21 12:43:30 UTC 2026 : Sending request to https://sqs.us-east-1.amazonaws.com/140023390772/orders_queue

Tue Apr 21 12:43:30 UTC 2026 : Received response. Status: 200, Integration latency: 31 ms

STATUS 200 MEANS THAT A **CUSTOMER'S ORDER** WAS SUCCESSFULLY **SENT** TO THE **SQS QUEUE**. ANY OTHER **STATUS CODE** MEANS THAT THE **ORDER** SENT INTO **SQS** FROM **API GATEWAY** WAS **UNSUCCESSFUL**.

XI) CHECK YOUR **EMAIL**. YOU SHOULD GET **AN EMAIL NOTIFICATION** FROM **SNS** IF YOU HAVE SUCCESSFULLY BUILT YOUR INFRASTRUCTURE. THE EMAIL SHOULD LOOK LIKE THE BELOW.

```
{"orderId": {"S": "2607c8e0-88a6-4f8e-b041-6bc9b3c87b87"}, "order": {"S": "{\"item\":  
\\\"noise canceling headset\\\",\\r\\n\\\"customerID\\\":\\\"12345\\\"}"}}
```

WHAT IT MEANS IS THAT THE **TEST DATA** WENT THROUGH THE **ENTIRE INFRASTRUCTURE** WE BUILT. YOU CAN CHECK FOR THE **LATEST CUSTOMER ORDER** IN THE **orders DYNAMO DB TABLE**.

TECH DEALER

DELIVERING TOMORROW'S TECH TODAY

